CPU/GPU/NPU-Aware RAG Pipeline on an Intel Core Ultra: Phase 1 Measurement and Profiling Report

Ting-Yu Yu, Cheng-Yang Wu, Jonas Tsai, Vincent Chen

CSE 291A, Fall 2026 – Phase 1 (due Nov 4)

1 Goal and summary

Our project asks where each stage of an edge RAG pipeline should run on a machine whose CPU, integrated GPU (iGPU) and NPU share one DRAM. The pipeline is: query → **S1** embedding (bge-small-en-v1.5) → vector search → **S2** rerank (bge-reranker-base) → **S3** generation (Qwen2.5-1.5B-Instruct, INT4). Each model is loaded once on one device. Phase 1 only measures: can every stage run on every device, how fast and at what cost, and why do devices differ.

Main findings.

- All 9 stage×device combinations run on the requested device, with no runtime fallback.
- The iGPU is the fastest device for S2 and S3 at every size we measured. The NPU is competitive only for small encoder inputs (S1 batch 1; S2 with 10 candidates) and degrades sharply for larger shapes. The CPU is 15–20× slower for S2 and for S3 prefill.
- End to end, all stages on the iGPU is best for latency, throughput up to ∼2.4 queries/s, and answer quality. Moving S2 to the NPU adds 110–140 ms per query when S3 is on the iGPU or CPU (nothing when S3 is also on the NPU), lowers recall@3 from 0.87 to 0.82–0.83, and gives at most a few percent more capacity.
- Placing S1 on any device changes end-to-end latency by about 20 ms or less; S1 is irrelevant to the decision.
- Running two devices at once slows both, because they share memory bandwidth: S3 decode on the iGPU slows by 58% while an NPU reranker runs.

2 Setup

Machine. An Intel Core Ultra 7 356H (16-core CPU, Arc iGPU and NPU, sharing 96 GB of DRAM) running OpenVINO 2026.4.1 with OpenVINO GenAI. The machine is shared with other users (load average 1–8 during runs).

Models. S1 and S2 are FP16 OpenVINO exports with static shapes (S1 $[b, 64]$, S2 $[k, 256]$, padded and truncated), because the NPU requires static shapes; the same shapes are used on all devices. S3 is INT4 with symmetric, channel-wise weights; a first asymmetric export produced garbage on the NPU. Decoding is greedy, at most 32 new tokens in the pipeline.

Data. A closed corpus of 25,880 Wikipedia passages (SQuAD train/dev and HotpotQA). Search is exact cosine similarity in numpy on the CPU (≈4 ms) and is not part of placement. Phase 1 uses the first 100 of 500 SQuAD-dev questions (20 candidates, top-3 passages to S3); 500 HotpotQA questions are held out for Phase 2.

Protocol. Stage sweeps vary one input at a time: S1 batch (1/8/32), S2 candidates (10–100, real candidates), S3 prompt (256/1k/4k) and output length (32/128/256). They report warm medians over 10 repetitions (S3: 5 on the iGPU, 3 on CPU and NPU); S3 uses a fresh prompt per repetition so the prefix cache cannot inflate results. End-to-end runs process 100 questions under a fixed mapping, sequentially or overlapped (S1/S2 of query i+1 with S3 of query i). Load runs use closed-loop clients (1/2/4) or Poisson arrivals (0.5–3 q/s). Contention runs measure S3 decode while S2 loops on another device.

Instrumentation (no root). *Placement*: `EXECUTION_DEVICES` for S1/S2; for all stages, the iGPU cycle counter (DRM fdinfo) and NPU busy counter around each call. *Memory*: a 20 ms sampler of RSS, iGPU GTT and NPU memory; footprint is load peak minus baseline. *Transfer*: S1/S2 calls are split into tokenize, set input, infer and get output, and infer is repeated with zero-copy tensors to estimate the input copy. *Cold start*: NPU driver and iGPU kernel caches disabled.

3 Support matrix and per-stage results

Table 1: Support and cost (Phase 1b). No fallback in any row: activity appeared only on the requested device. Load is warm cache / true cold.

Stage	Device	Load (s)	Memory (MB)
S1	CPU	0.39 / 0.39	host
	iGPU	0.44 / 1.0	130
	NPU	0.07 / 1.0	67
S2	CPU	3.3 / 3.3	host
	iGPU	3.1 / 3.8	596–818
	NPU[a]	0.12 / 68	726
S3	CPU[c]	0.67 / 0.67	host
	iGPU[c]	1.6 / 3.0	1002
	NPU[bc]	4.9 / 10.7	1355–2030

Table 2: S1 and S2 warm median latency (ms), Phase 1b run, 10 repetitions.

S1 batch	1	8	32
CPU	7.8	52.8	202
iGPU	1.4	4.2	15.1
NPU	2.4	9.0	27.9

S2 cand.	10	20	50	100
CPU	1076	2092	5180	10319
iGPU	71	129	276	522
NPU	84	173	434	14200

[a]Static shapes only: 50 candidates take 241 s to compile and 100 candidates 21 min. [b]Needs symmetric INT4 weights (asymmetric INT8 compiles but generates garbage); load for a 1152-token maximum prompt; memory for `MAX_PROMPT_LEN` 384–4224. [c]`LLMPipeline` does not report `EXECUTION_DEVICES`; placement rests on the activity counters.

All nine pairs run with no silent CPU fallback. The NPU is the least flexible device: it needs static shapes, including a maximum prompt length for the LLM, and compile cost grows quickly with shape. The compile cache hides this after the first load (reranker 0.12 s warm, 68 s cold); iGPU cold loads cost only 0.5–1.4 s more.

S1. All devices take under 10 ms at batch 1, where the iGPU and NPU are close (iGPU 1.4–2.9 ms depending on the run, NPU 2.4 ms) and fixed overhead dominates. The iGPU scales best and the CPU worst. **S2.** The iGPU is the best choice from 10 candidates up. The NPU matches it only at 10 candidates, is 1.6× slower at 50, and collapses at 100 (14 s).

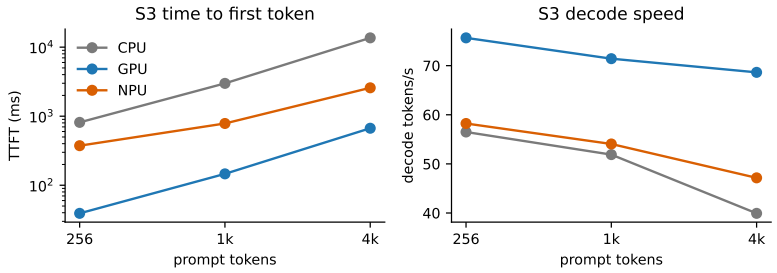


Figure 1: S3 on synthetic prompts (128 output tokens, fresh prompt per repetition; the text quotes the Phase 1b rerun, within 3%). NPU compiled for `MAX_PROMPT_LEN`=4224.

S3. Prefill is the large difference (Fig. 1): at 1k tokens TTFT is 146 ms on the iGPU, 771 ms on the NPU and 2932 ms on the CPU. Decode speed is 69–75 tokens/s on the iGPU, 47–58 on the NPU and 40–56 on the CPU; decode is memory-bandwidth bound, so the gap is much smaller than in prefill. On the iGPU, output length does not change TTFT or time per output token (TPOT 13.3–14.6 ms), so total time is linear in output tokens. The NPU's TTFT depends on the compiled prompt limit rather than the real prompt: a 256-token prompt takes 99.6 ms with `MAX_PROMPT_LEN`=384 but 366 ms with 4224, and larger limits use more memory (Table 1).

Memory. An uncached NPU compile needs 1.7–3.2 GB of host RAM and an end-to-end process 1.7–2.4 GB of RSS, so memory is not a constraint here.

Data movement. With shared DRAM, "transfer" is a host-side copy, and payloads are small (per query: 1.5 KB query vector, 14.6 KB candidate text, 2.4 KB prompt). The iGPU reranker's input copy is 3–13 ms and the NPU's under 0.4 ms. Reranker tokenization (8–35 ms, 10–15% of iGPU S2) is a larger host-side cost; S3 tokenization and detokenization take under 2 ms. The first inference after loading is up to 2× slower (NPU S2: 199 versus 84 ms).

4 End-to-end results

Table 3: All 27 mappings were measured; S1 shown on the iGPU (other S1 devices change latency by at most 22 ms). Sequential; n = questions measured.

S2	S3	Median (ms)	p90 (ms)	q/s	recall@3	F1	n
iGPU	iGPU	**437**	621	**2.23**	0.87	0.36	300
NPU	iGPU	573	757	1.73	0.82	0.34	300
CPU	iGPU	2437	2624	0.41	0.87	0.36	300
iGPU	NPU	843	1093	1.19	0.87	0.38	200
NPU	NPU	825	1103	1.19	0.82	0.32	100
CPU	NPU	2831	3091	0.35	0.87	0.38	100
iGPU	CPU	1866	2445	0.52	0.87	0.34	100
NPU	CPU	1991	2423	0.51	0.82	0.32	100
CPU	CPU	3845	4384	0.26	0.87	0.34	100

With S3 on the iGPU, a query takes 437 ms: S1 2 ms, search 4 ms, S2 118 ms, S3 316 ms (TTFT 81 ms). Putting S3 on the NPU doubles the latency, and on the CPU it multiplies it by 4. S2 on the NPU does not help sequentially (231 ms versus 118 ms) because the NPU reranker is slower than the iGPU's; with overlapped execution, where S2 of the next query runs while S3 decodes, S2 on the NPU reaches 2.45–2.50 queries/s versus 2.38 for all-iGPU, a gain of about 5%. All-iGPU gains only 7% from overlap because both stages share one device; S2 on the NPU gains 44% over its own sequential run, while S3 on the iGPU slows from 316 to about 385 ms (Sec. 4.2). Placement also affects output: the NPU reranker reorders candidates (recall@3 0.87 → 0.82–0.83), while S2 on the CPU and iGPU rank identically. Real RAG prompts are shorter than the 1k/4k sweep points: top-1/3/8 passages give 213/555/1379 tokens (max 1888), and top-8 takes 764 ms end to end on the iGPU (TTFT 247 ms) and 1342 ms with S3 on the NPU. More passages raise recall (0.74/0.87/0.90) but lower F1 (0.40/0.36/0.25 on the iGPU), so the 1.5B model does worse with longer context; we have not analyzed why.

4.1 Load

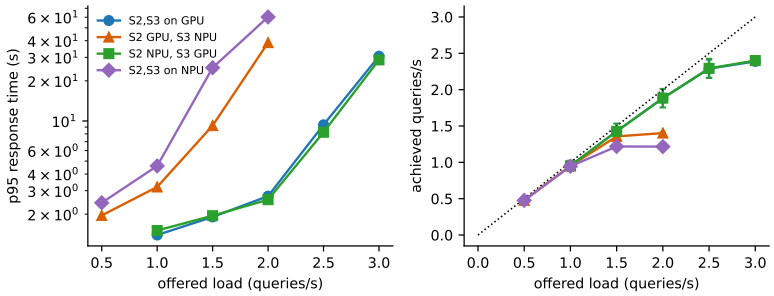


Figure 2: Open-loop Poisson arrivals (3 rounds, at least 150 queries per point; error bars: std over rounds). Response time includes queueing.

Every mapping behaves like a queue with a hard capacity (Fig. 2): below it, response time is close to the service time; above it, waiting grows without bound (p50 of 42 s at 2.0 q/s for S2 and S3 on the NPU). Capacity is 2.4 q/s for S3 on the iGPU and 1.2–1.4 for S3 on the NPU. Between the two S3-on-iGPU mappings the evidence is mixed: in closed-loop runs S2 on the NPU gives 2.45–2.49 q/s versus 2.38 (+4%), but under Poisson arrivals the two have the same capacity within noise (2.39 vs 2.40, std 0.12–0.14 q/s), the same p50 at 2.0 q/s, and a 12% lower p95 at 2.5 q/s for the NPU variant. At 1 q/s all-iGPU has the lower median (0.61 versus 0.73 s). Hence any benefit of an NPU reranker appears only near saturation and is small.

4.2 Why concurrent devices interfere

Running S3 decode next to a reranker on another device (Table 4) causes the same slowdown from a thread or a separate process, and none from a tokenize-only thread, so the cause is not the Python GIL but shared memory

bandwidth: iGPU TPOT rises 58% next to an NPU reranker and NPU TPOT 70% next to an iGPU reranker, while a CPU reranker costs only 8%. This is why overlapped pipelines gain much less than the sum of their parts.

Table 4: S3 decode (1k-token prompt, 128 output tokens) with an S2 reranker running elsewhere.

S3 device	Aggressor	Mode	TTFT (ms)	TPOT (ms)
iGPU	none	–	148	14.0
iGPU	rerank on NPU	thread / process	191	22.1
iGPU	tokenize only	thread	157	14.2
iGPU	rerank on CPU	process	162	15.2
NPU	none	–	769	18.6
NPU	rerank on iGPU	process	1052	31.5

5 Device preference (inputs to the Phase 2 policy)

Table 5: Which device under which condition (Phase 1 conclusion).

Stage	Preferred device	Conditions and exceptions
S1	iGPU, but any device works	Under 10 ms per query everywhere; avoid the CPU for batches of 8 or more.
S2	iGPU alone; NPU only when overlapping S3	NPU only up to 50 candidates with a precompiled shape, and it costs about 4 points of recall@3; never the CPU.
S3	iGPU	NPU is about 1.9× slower end to end, with TTFT tied to its compiled maximum prompt; CPU only as a fallback.
Pipeline	All-iGPU; S2 on the NPU may help only near saturation	Same Poisson capacity (≈2.4 q/s) within noise; S2 on the NPU gives +4% closed-loop throughput and 12% lower p95 at 2.5 q/s; iGPU+NPU overlap costs up to +58% S3 TPOT.

6 Limitations

- **Two NPU models in one process.** An NPU LLM together with any other NPU model in one process fails at first inference (`zeMemAllocHost`, null handle), although the same configuration ran on Oct 27. For those mappings the LLM runs in its own process, which adds 0.4–0.6 ms per call and no measurable end-to-end overhead (842 vs 845 ms in a control).
- **Noise and sample size.** The machine is shared (load average 1–8). Only the main 9 S3-on-iGPU mappings have three rounds; the others have one or two. Quality comparisons use 100 questions with no confidence intervals. The S3 sweep on CPU and NPU covers only 128 output tokens, and its prompts are synthetic.
- **Scope.** The corpus is small (25,880 passages), so search is negligible; with a much larger index it would matter. Energy was not measured because power counters need root access, so our comparison is performance only.

A Timeline

Oct 27–Nov 2	*Done:* models, corpus, stage sweeps, instrumentation, all 27 mappings, load and contention runs, NPU workaround
Nov 4	**Phase 1 report**
Nov 5–13	Placement policy (lookup table / performance model from the profiles); policy decision overhead
Nov 14–20	Baselines (fixed mappings, best static on dev); held-out HotpotQA and mixed short/long prompts
Nov 21–27	Policy under load, including switching and transfer cost; S2 length buckets; per-class NPU `MAX_PROMPT_LEN`
Nov 28–Dec 3	Qwen2.5-3B sensitivity check; confidence intervals for quality differences
Dec 4–10	**Final report** (due Dec 10)